📘 FILE HANDLING — SHORT REVISION NOTES

1. What is File Handling?

File handling means using Python to work with files.

```
OPEN → READ / WRITE / MODIFY → CLOSE
```

Real-world examples:

```
Anime Subtitle Project
.srt → Read subtitles

Vocabulary System
.txt / .json → Save learned words

Smart File Organizer
Files → Detect → Create folders → Move files
```

---

2. open ( ) Function

Basic syntax:

```
file = open("file_name.txt", "mode")
```

Example:

```
file = open("notes.txt", "r")
```

Meaning:

```
"notes.txt" → Which file?
"r"         → What do you want to do?
file        → Variable representing the opened file
```

---

3. File Modes

Mode	Meaning	File Missing	Existing Content
`"r"`	Read	❌ Error	Safe
`"w"`	Write	✅ Creates	⚠️ Overwrites
`"a"`	Append	✅ Creates	Adds at end
`"x"`	Create	✅ Creates	❌ Error if exists
`"r+"`	Read + Write	❌ Error	Does not automatically erase
`"w+"`	Write + Read	✅ Creates	⚠️ Overwrites
`"a+"`	Append + Read	✅ Creates	Adds at end
`"b"`	Binary mode	Depends	Used with another mode

`"t"`	Text mode	Default	Normal text

Memory trick

```
r → Read
w → Write + overwrite
a → Add at end
x → Create only if file doesn't exist
```

---

4. Reading Files

`read()`

Reads the entire remaining content.

```
with open("notes.txt", "r", encoding="utf-8") as file:
    data = file.read()

print(data)
```

Returns:

`String`

`read(n)`

Reads a specific number of characters.

```
file.read(5)
```

Example:

```
Hello Python
```

Output:

```
Hello
```

`readline()`

Reads one line at a time.

```
line = file.readline()
```

Returns:

`String`

Every call moves to the next line.

`readlines()`

Reads all remaining lines and returns a list.

```
lines = file.readlines()
```

Example result:

```
["Apple\n", "Banana\n", "Mango\n"]
```

Reading with a loop

Professional and memory-friendly for large files:

```
with open("notes.txt", "r", encoding="utf-8") as file:
    for line in file:
        print(line)
```

Useful for:

```
Large logs
Subtitle files
CSV files
Large text files
```

---

5. File Pointer

A file pointer is the **current position where Python is reading or writing**.

Example:

```
Hello Python
^
Pointer starts here
```

After:

```
file.read(5)
```

The pointer moves forward.

```
Hello Python
     ^
```

Important behavior

```
file.read()
file.read()
```

The second `read()` may return an empty string because the pointer is already at the end.

---

6. `tell()`

`tell()` tells you the current position of the file pointer.

```
print(file.tell())
```

Think:

```
tell() → "Where am I?"
```

---

7. `seek()`

`seek()` moves the file pointer.

```
file.seek(0)
```

Moves the pointer back to the beginning.

Think:

```
seek() → "Move me"
tell() → "Where am I?"
```

---

8. Writing Files

`write()`

Writes a string.

```
with open("notes.txt", "w", encoding="utf-8") as file:
    file.write("Hello Python")
```

Multiple writes:

```
file.write("Hello")
file.write("World")
```

Output:

```
HelloWorld
```

Python does **not automatically add a new line**.

Use:

```
file.write("Hello\n")
file.write("World\n")
```

`writelines()`

Writes multiple strings.

```
lines = [
    "Apple\n",
    "Banana\n",
    "Mango\n"
]

file.writelines(lines)
```

⚠️ `writelines()` does **not automatically add `\n`**.

---

9. Closing Files

Traditional method:

```
file = open("notes.txt", "r")

data = file.read()

file.close()
```

`close()` releases the opened file resource.

---

10. `with open()` — Recommended Way

```
with open("notes.txt", "r", encoding="utf-8") as file:
    data = file.read()
```

After the block finishes, Python automatically closes the file.

Even if an exception occurs inside the `with` block, Python still performs cleanup.

Professional preference

```
with open(...) as file:
```

is generally preferred over manually calling:

```
file.close()
```

---

11. Encoding — `utf-8`

Computers store data as numbers/binary.

Encoding tells Python:

"How should these bytes be converted into readable text?"

Example text:

```
Hello
हिंदी
日本語
한국어
```

Using:

```
encoding="utf-8"
```

helps Python correctly handle Unicode text.

Professional example:

```
with open("notes.txt", "r", encoding="utf-8") as file:
    data = file.read()
```

For your subtitle project, this is especially useful because subtitle files may contain different languages.

---

12. Text vs Binary Mode

Text mode

```
open("notes.txt", "r")
```

Used for readable text:

```
.txt
.json
.csv
.srt
```

Binary mode

```
open("image.png", "rb")
```

Used for raw bytes:

```
.jpg
.png
.pdf
.exe
```

Common combinations:

```
"rb" → Read binary
"wb" → Write binary
"ab" → Append binary
```

---

13. File Paths

Simple relative path:

```
open("notes.txt")
```

File inside a folder:

```
open("data/notes.txt")
```

14● `pathlib` — Professional Path Handling

Instead of manually joining paths:

```
"data/notes.txt"
```

Use:

```
from pathlib import Path

file_path = Path("data") / "notes.txt"
```

Benefits:

```
Readable
Cross-platform
Easy path operations
Easy folder/file handling
```

```
Professional project structure
```

Then:

```
with open(file_path, "r", encoding="utf-8") as file:
    data = file.read()
```

Or:

```
with file_path.open("r", encoding="utf-8") as file:
    data = file.read()
```

---

15. Creating Folders

```
from pathlib import Path

folder = Path("data")

folder.mkdir()
```

If it already exists:

```
folder.mkdir(exist_ok=True)
```

⚠️ Correct spelling:

```
exist_ok=True
```

Not:

```
exists_ok=True
```

---

16. Checking Whether a File Exists

```
from pathlib import Path

file_path = Path("notes.txt")

print(file_path.exists())
```

Returns:

```
True
```

or:

```
False
```

Other useful methods:

```
file_path.is_file()
file_path.is_dir()
```

---

17. Advanced Modes — `r+`, `w+`, `a+`

`r+`

```
Read + Write
File must already exist
Does not automatically erase content
open("notes.txt", "r+")
```

⚠️ Writing happens from the current pointer position.

`w+`

```
Read + Write
Creates file if missing
Overwrites existing content
open("notes.txt", "w+")
```

⚠️ Dangerous if you need old data because it clears the file first.

`a+`

```
Append + Read
Creates file if missing
Writing always happens at the end
open("notes.txt", "a+")
```

If you want to reread from the beginning:

```
file.seek(0)
```

---

18. JSON Files

A normal text file:

```
beautiful
```

```
journey
mysterious
```

A JSON file can store structured data:

```
{
    "beautiful": "attractive",
    "journey": "a trip"
}
```

Python:

```
import json

with open("vocabulary.json", "r", encoding="utf-8") as file:
    data = json.load(file)
```

Save:

```
import json

with open("vocabulary.json", "w", encoding="utf-8") as file:
    json.dump(data, file, indent=4)
```

Useful for your projects:

```
Settings
Vocabulary
User data
Application data
Progress tracking
```

---

19. CSV Files

CSV means:

```
Comma-Separated Values
```

Example:

```
word,meaning
beautiful,attractive
journey,trip
```

Useful for:

```
Excel-like data
Reports
Student records
Vocabulary exports
```

Python provides the `csv` module for this.

---

20. Professional File Handling Pattern

Basic:

```
file = open("notes.txt", "r")
data = file.read()
file.close()
```

Better:

```
with open("notes.txt", "r", encoding="utf-8") as file:
    data = file.read()
```

Professional project style:

```
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent
DATA_DIR = BASE_DIR / "data"

DATA_DIR.mkdir(exist_ok=True)

FILE_PATH = DATA_DIR / "notes.txt"

with FILE_PATH.open("r", encoding="utf-8") as file:
    data = file.read()
```

Mental model:

```
BASE_DIR → Where is my Python project?

DATA_DIR → Where should application data go?

FILE_PATH → What exact file am I working with?
```

⚠️ COMMON MISTAKES

1. Wrong mode

```
open("data.txt", "r")
```

then:

```
file.write("Hello")
```

❌ Error — read mode cannot write.

---

2. Accidentally using `"w"`

```
open("important.txt", "w")
```

⚠️ Existing content can be erased immediately when opened in write mode.

Use `"a"` if you want to preserve existing content and add more.

---

3. Forgetting `\n`

```
file.write("Apple")
file.write("Banana")
```

Result:

```
AppleBanana
```

Use:

```
file.write("Apple\n")
```

---

4. Forgetting the file pointer

```
data = file.read()
print(file.read())
```

The second read may be empty because the pointer reached the end.

Fix:

```
file.seek(0)
```

---

5. Wrong `mkdir()` argument

❌ Wrong:

```
folder.mkdir(exists_ok=True)
```

✅ Correct:

```
folder.mkdir(exist_ok=True)
```

---

6. Wrong encoding argument

❌ Wrong:

```
encoding="utf=8"
```

✅ Correct:

```
encoding="utf-8"
```

---

7. Trying to read a missing file

```
open("missing.txt", "r")
```

Result:

```
FileNotFoundError
```

Later, Error Handling will teach you how to handle this safely.

🎤 COMMON INTERVIEW QUESTIONS

Beginner

1. What is file handling?
2. What is the difference between `r`, `w`, and `a`?
3. What happens if a file doesn't exist in `"r"` mode?
4. What happens when `"w"` opens an existing file?
5. Difference between `read()`, `readline()`, and `readlines()`?
6. What does `with open()` do?
7. Why should files be closed?
8. What is `encoding="utf-8"`?

Intermediate

9. What is a file pointer?
10. Difference between `tell()` and `seek()`?
11. What is the difference between `writelines()` and `write()`?
12. Difference between text and binary mode?
13. What are `r+`, `w+`, and `a+`?
14. Why might reading a large file line by line be preferable?

Project Level

15. Why use `pathlib` instead of hard-coded paths?
16. How do you safely handle a missing file?
17. How would you store application data: TXT, JSON, CSV, or a database?

18. How would you avoid accidentally overwriting important data?
19. How would you organize file paths in a professional Python project?